

Assignment 8

Aathish Sivasubrahmanian 3122215001001

Objective:

To apply Ensemble techniques such as Bagging, Random Forest and AdaBoost on the input dataset and perform classification.

Code:

```
# %%
#libraries:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA

from sklearn.utils import shuffle
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split
from numpy import mean
from numpy import std
from sklearn import svm
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis as LDA
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from matplotlib import pyplot
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import RidgeClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import cross_val_score,
cross_val_predict

# %%
df=pd.read_csv("/home/ml1ab1/Downloads/archive/data.csv")
df.head()
```

```

# %%
df.describe()#data description

sns.countplot(x = df['diagnosis'], data=df, palette='hls')
plt.xticks(rotation=90)
plt.show()

# %%
df = df.replace({'diagnosis': {'M': 1,
                                'B': 0}})
df

# %%
#Matrix Covariace information:
correlation_matrix = df.corr().round(2)
correlation_matrix

# %%
mask = np.triu(np.ones_like(df.corr()))
dataplot = sns.heatmap(df.corr(),mask=mask)

# displaying heatmap
plt.show()

# %%
!pip install pca

# %%
from pca import pca
from sklearn.decomposition import PCA
# Initialize
model = pca()
train_x = df.iloc[:,2: -1]
train_y = df['diagnosis']
# Fit transform
out = model.fit_transform(train_x, train_y)

# Print the top features. The results show that f1 is best,
followed by f2 etc\
print('_____')
print('Feature importance by PCA')
print(out['topfeat'])

# %%
X_train, X_test, y_train, y_test = train_test_split(train_x,
train_y, test_size = 0.3, random_state = 0)

# %%
model =
RandomForestClassifier(class_weight="balanced",random_state=0)
k = 5

```

```

scoring = 'accuracy'

#use k-fold cross validation to evaluate model on training data
scores = cross_val_score(model, X_train, y_train, cv=k,
scoring=scoring)
print(f"Accuracy scores for {k} folds: {scores}")
print(f"Mean accuracy score: {np.mean(scores)}")

#use k-fold cross validation to generate predictions on test data
predictions = cross_val_predict(model, X_test, y_test, cv=k)
print(f"Accuracy score on test data: {accuracy_score(y_test,
predictions)}")

#calculate precision and recall scores
print(f"Precision score on test data: {precision_score(y_test,
predictions)}")
print(f"Recall score on test data: {recall_score(y_test,
predictions)}")

#create and plot confusion matrix
cm = confusion_matrix(y_test, predictions)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()

#print coefficients of the model
model.fit(X_train,y_train)

# %%
importance = model.feature_importances_
# summarize feature importance
for i,v in enumerate(importance):
    print('Feature: %0d, Score: %.5f' % (i,v))
# plot feature importance
pyplot.bar([x for x in range(len(importance))], importance)
pyplot.show()

#Confusion Matrix
cm4 = confusion_matrix(y_test, predictions,
labels=model.classes_)
disp4 = ConfusionMatrixDisplay(confusion_matrix=cm4,
display_labels=model.classes_)

disp4.plot()
plt.show()

print(f"Precision : {precision_score(y_test, predictions)}")
print(f"Recall : {recall_score(y_test, predictions)}")

```

Inference:

The given model had a precision of 0.9032258064516129 and a Recall of 0.8888888888888888

Learning Outcome:

Applied Ensemble techniques such as Bagging, Random Forest and AdaBoost on the input dataset and performed classification.